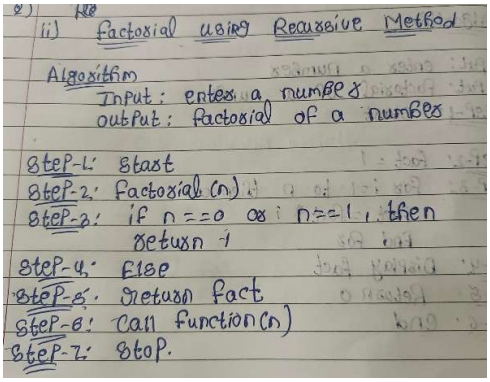
Practical - 04

Aim: Implementation and Time analysis of factorial program using iterative and recursive method

Recursive method -

Algorithms:


ii) Factorial using Recursive Method

Algorithm
Input: enter a number
output: factorial of a number

Step-1: Start
Step-2: Factorial (n)
Step-3: if n==0 or n==1, then
return 1
Step-4: Else
Step-5: Return fact
Step-6: Call function(n)
Step-7: Stop.

Code:-

```
#include <iostream>
using namespace std;

int factorial(int n)
{
   if (n == 0 || n == 1)
      return 1;

   return n * factorial(n - 1);
}

int main()
```

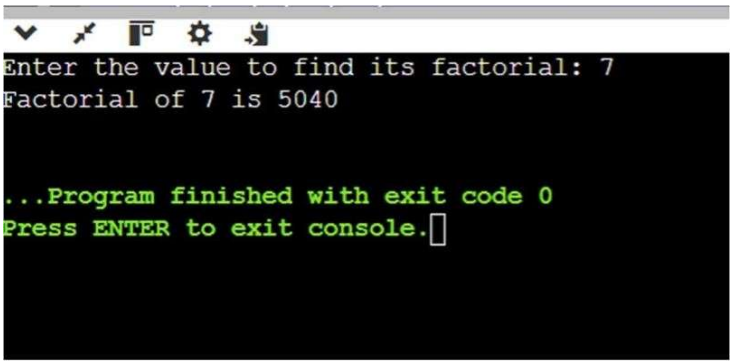
```
{
    int n;

    cout << "Enter a number: ";
    cin >> n;

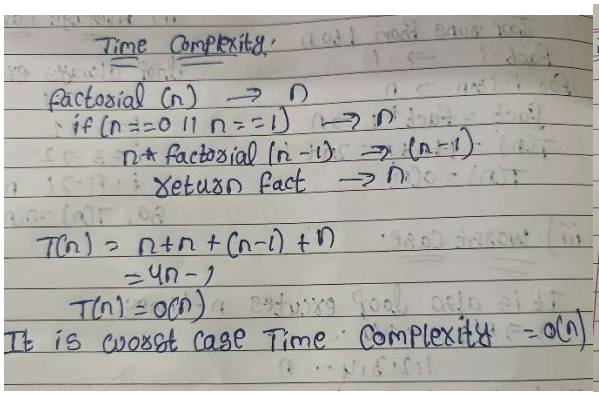
    cout << "Factorial = " << factorial(n) << endl;

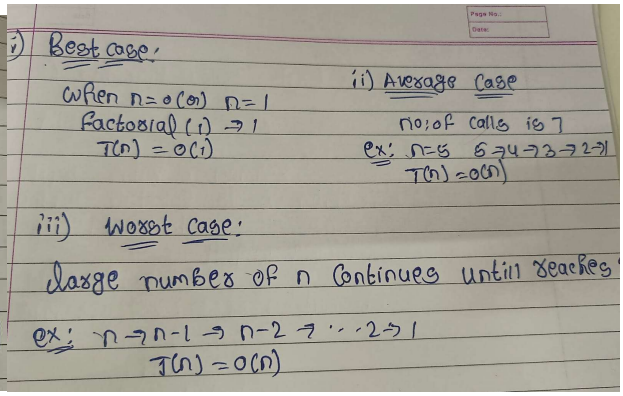
    return 0;
}
```

Output:-



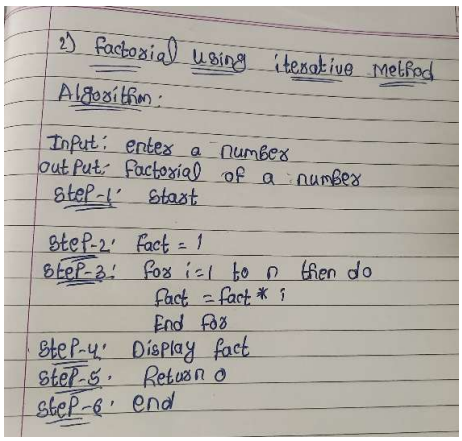
Time complexity:-





Iterative method -

Algorithms:



Code:

```
#include <iostream>
using namespace std;

int factorial(int n)
{
   int fact = 1;

   for (int i = 1; i <= n; i++)
   {
      fact = fact * i;
   }

   return fact;
}

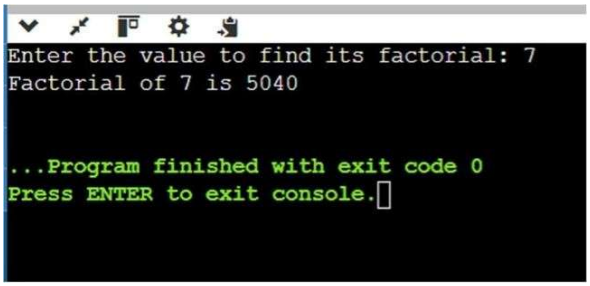
int main()
{
   int n;
```

```
    cout << "Enter a number: ";
    cin >> n;

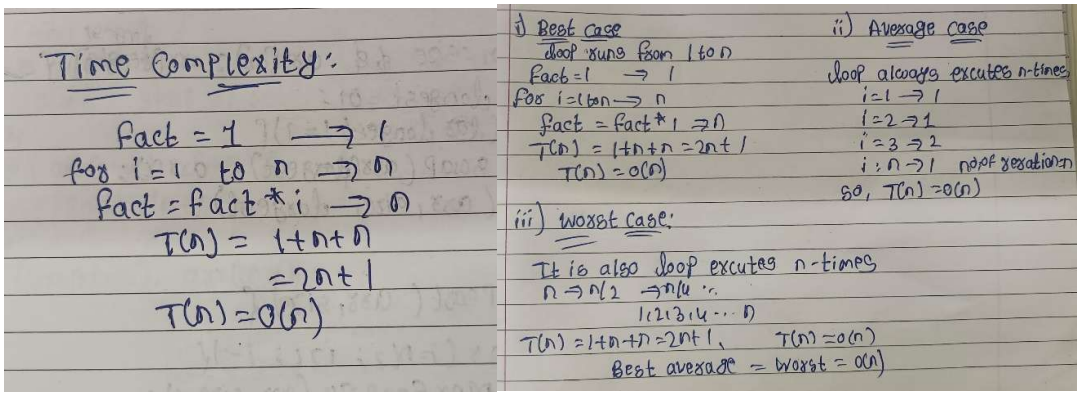
    cout << "Factorial = " << factorial(n) <<
endl;

    return 0;
}
```

Output:



Time complexity:



Conclusion:-

The factorial program was implemented using both iterative and recursive methods. Both approaches produce the same result, but they differ in their execution and memory requirements.

The iterative method uses a loop to calculate the factorial and has a time complexity of O(n) and a space complexity of O(1). Therefore, it is generally more memory-efficient and suitable for larger values of n.